



SCHOOL *of* DATA SCIENCE

Catalyst Optimizer for Spark SQL

.....

Distributed Computing
School of Data Science
University of Virginia

Last updated: September 14, 2026

The Engine Driving Spark SQL

Spark SQL is one of the most advanced modules in Spark

It comprises most of the codebase

The Catalyst optimizer is at the heart of Spark SQL

It uses advanced Scala features

The Engine Driving Spark SQL, contd.

The Catalyst optimizer operates on Spark Dataframes as well

For example if we run

```
df.filter(df.close > 500).select("close").groupBy().avg("close")
```

Spark doesn't immediately execute each DataFrame operation

It starts with a logical plan which Catalyst optimizes

Trees are the Main Data Type

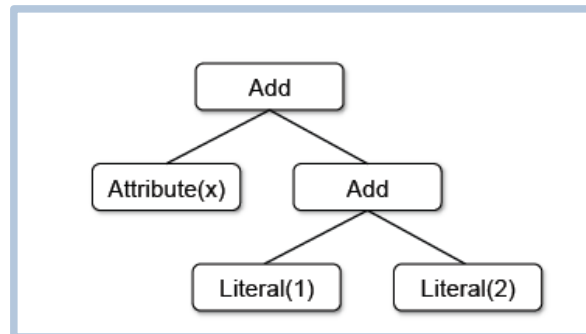
- > Catalyst uses a general library representing trees, applying rules to manipulate them
- > Trees are composed of node objects
- > Each node has a type, 0+ children
- > Immutable, manipulated using functional transformations

Trees for Evaluating Expressions

Simple example with three node classes:

- `Literal(value: Int)`: a constant value
- `Attribute(name: String)`: an attribute from an input row, e.g., "x"
- `Add(left: TreeNode, right: TreeNode)`: sum of two expressions.

Can build a tree for an expression like: $x + (1 + 2)$



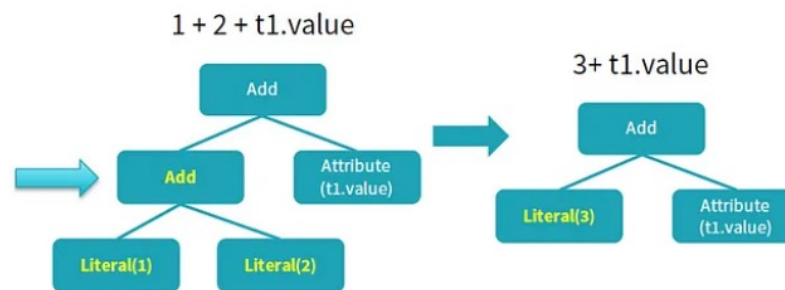
Rules

Manipulate trees with rules

Rules are functions that map a tree to another tree

Transform

```
val expression: Expression = ...  
expression.transform {  
  case Add(Literal(x, IntegerType), Literal(y, IntegerType)) =>  
    Literal(x + y)  
}
```



Transformations

One transform call can match multiple patterns:

```
tree.transform {  
  case Add(Literal(c1), Literal(c2)) => Literal(c1+c2)  
  case Add(left, Literal(0)) => left  
  case Add(Literal(0), right) => right  
}
```

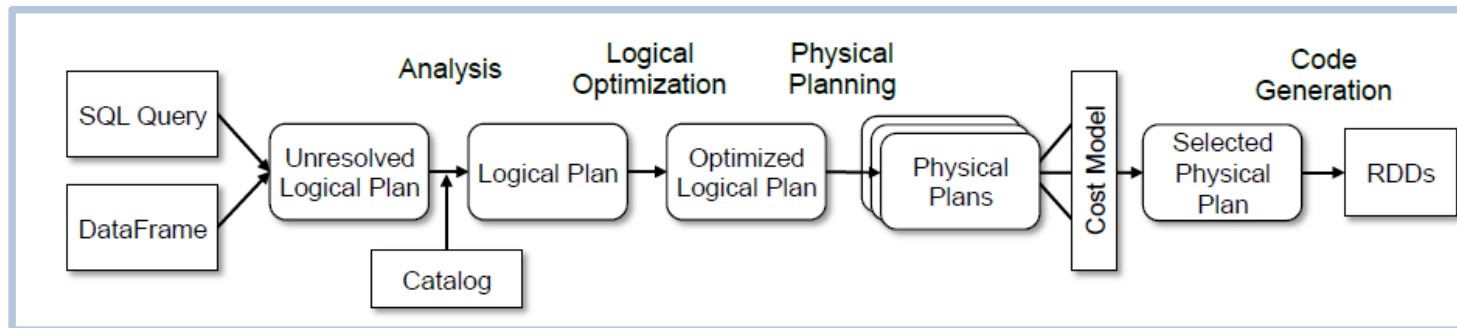
In practice, rules may need to execute multiple times to fully transform a tree.

Catalyst groups rules into batches, executing each batch until tree stops changing (fixed point).

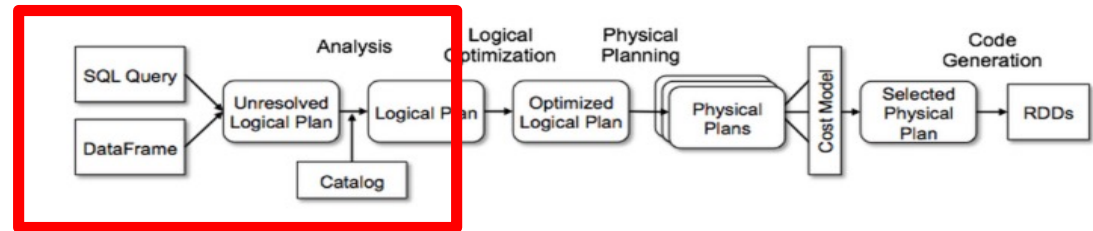
Transformation Framework

Catalyst implements transformations in four phases:

- 1) Analysis of logical plan
- 2) Logical plan optimization
- 3) Physical planning
- 4) Code generation to compile query to Java bytecode



Analysis



Begins with relation to be computed from SQL parser or DataFrame

Consider this query:

```
SELECT close
FROM prices
WHERE close > 500
```

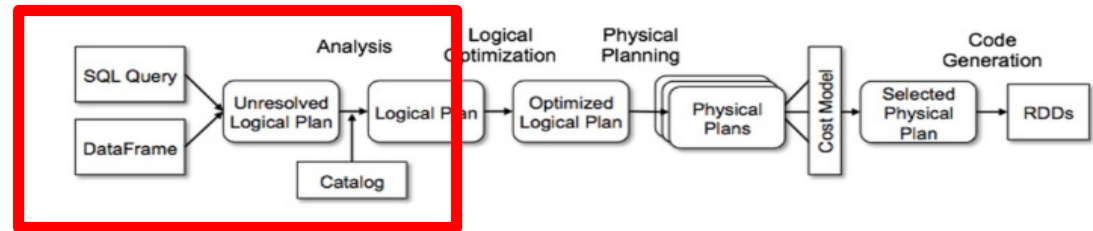
Where is **prices**? Where is close?

What is the data type of **close**?

close is *unresolved*

Need to look at table to resolve

Analysis



Example query:

```
SELECT close
FROM prices
WHERE close > 500
```

Catalyst analysis
does this:

1. "prices"
↓
Find prices in Catalog
2. "close"
↓
Find close in prices' schema
3. Give it an internal identity
↓
close#123

From Code to Logical Plan

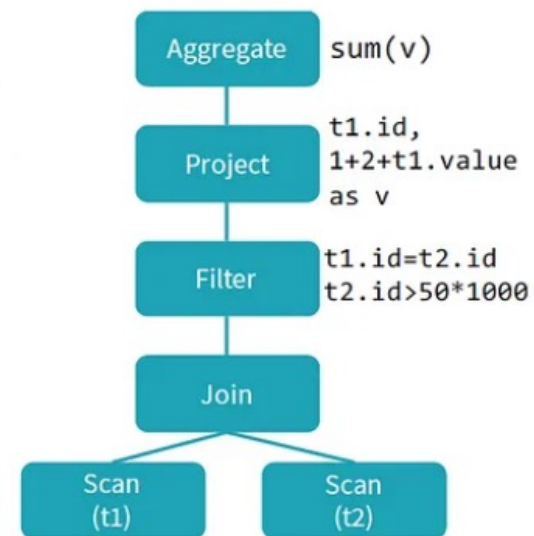
SQL Code

```
SELECT sum(v)
FROM (
  SELECT
    t1.id,
    1 + 2 + t1.value AS v
  FROM t1 JOIN t2
  WHERE
    t1.id = t2.id AND
    t2.id > 50 * 1000) tmp
```

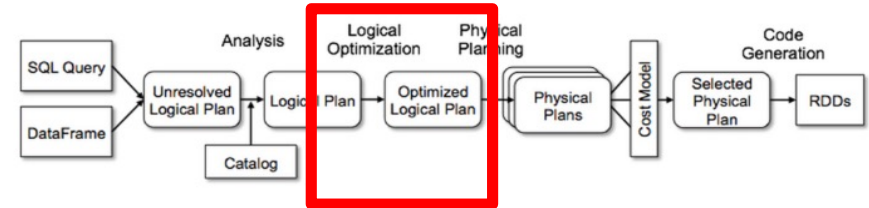
Logical Plan describes compute on datasets without defining how to do compute

Physical Plan defines the computation

Logical Plan



Plan Optimization



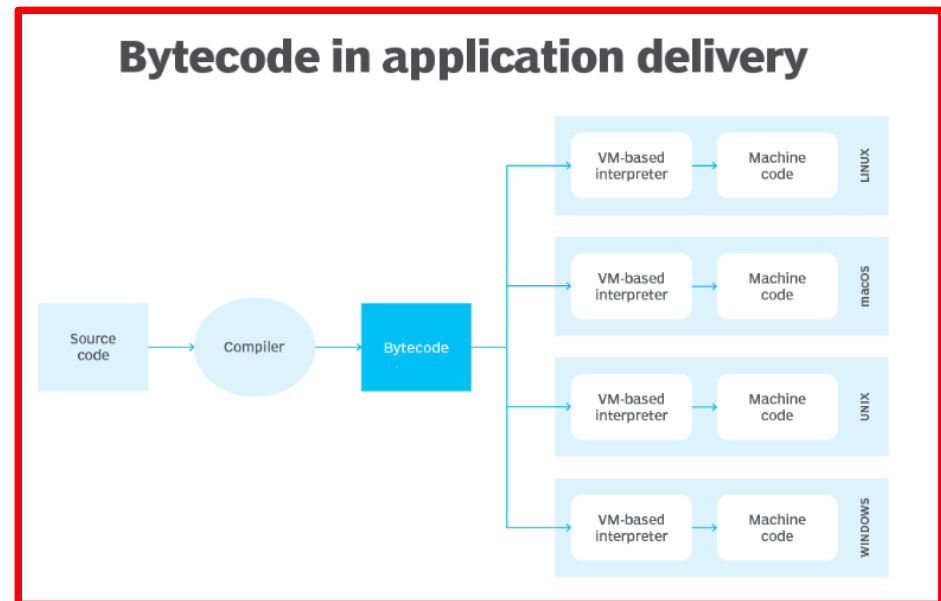
- > Logical optimization applies rule-based optimizations to logical plan (e.g., constant folding, predicate pushdown, Boolean expression simplification)
- > Generate several plans
- > Apply cost model to each, based on physical plan
- > Select the lowest cost plan

Background for Compute Optimization

Bytecode

Bytecode is code that sits between **source code** (human readable) and **machine code**

Bytecode is platform independent (runs on any OS)

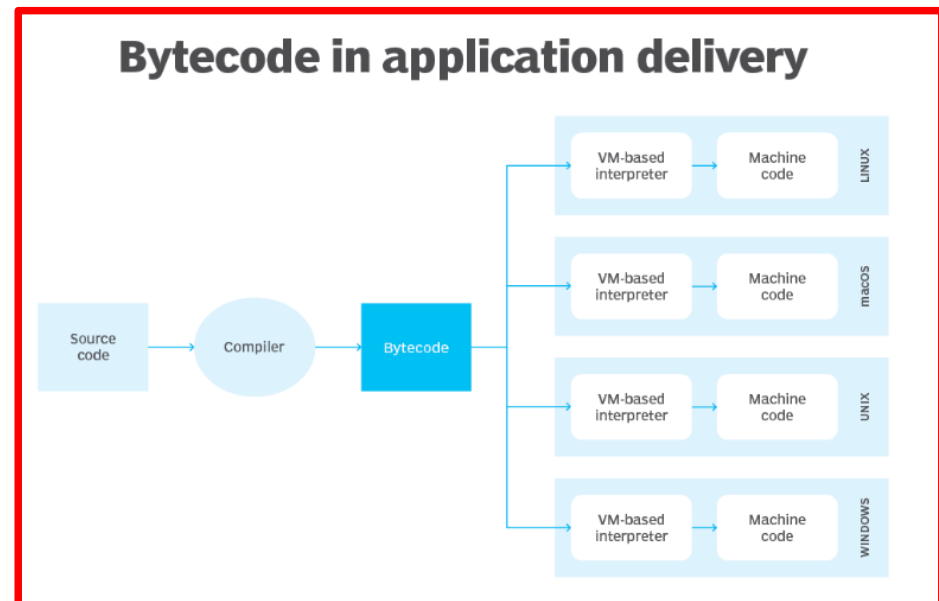


Java Virtual Machine (JVM)

JVM is software that takes **bytecode** and compiles to machine code

It can run any language that can be compiled to bytecode including:

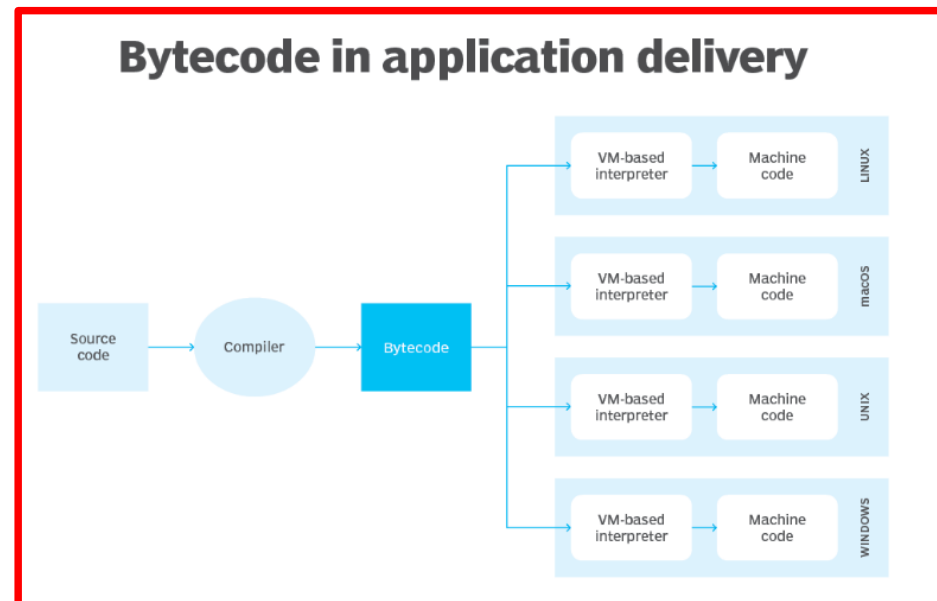
- > Java
- > **Scala**
- > Kotlin
- > Groovy



Java Virtual Machine (JVM)

After JVM compiles bytecode to machine code, machine code executes on CPU

Note: a specialized compiler generates machine code for GPU



Why is Scala Native in Spark?

At the time of Spark development, the competitor was Hadoop (for running Map Reduce)

Hadoop was coded in Java and runs on JVM

Java code can be quite verbose

Developers decided to use Scala:

- many engineers could develop in Scala
- less coding required
- compiles to bytecode for running on JVM

Compile Time vs Run Time

Compile time:

When programming code (source code) converted to machine code (executable code). This is done by *compiler*. Example representation: **111010111111110**

- Compilers are optimized for hardware architecture.
- Processing at this step is often much faster vs runtime.

Compile Time vs Run Time

Run time:

The time a program is running alongside all external instructions needed for proper execution

Follows compile time

Not as fast to run code at this stage

Specific Optimizations

Constant Folding

Sometimes code includes simple calculations w constants (*constant expressions*):

`seconds_in_day = 60 x 60 x 24`

`x = (2 + 3) * y`

The **compiler** can identify and evaluate the expressions.

Spark SQL will recognize and allow for evaluating constant expressions
at compile time rather than computing at runtime (faster!)

`seconds_in_day = 60 x 60 x 24` → `seconds_in_day = 86400`

`x = (2 + 3) * y` → `x = 5 * y`

Predicate Pushdown

Predicate is condition on query that returns true or false,
Usually in WHERE clause

Predicate pushdown filters data in database query

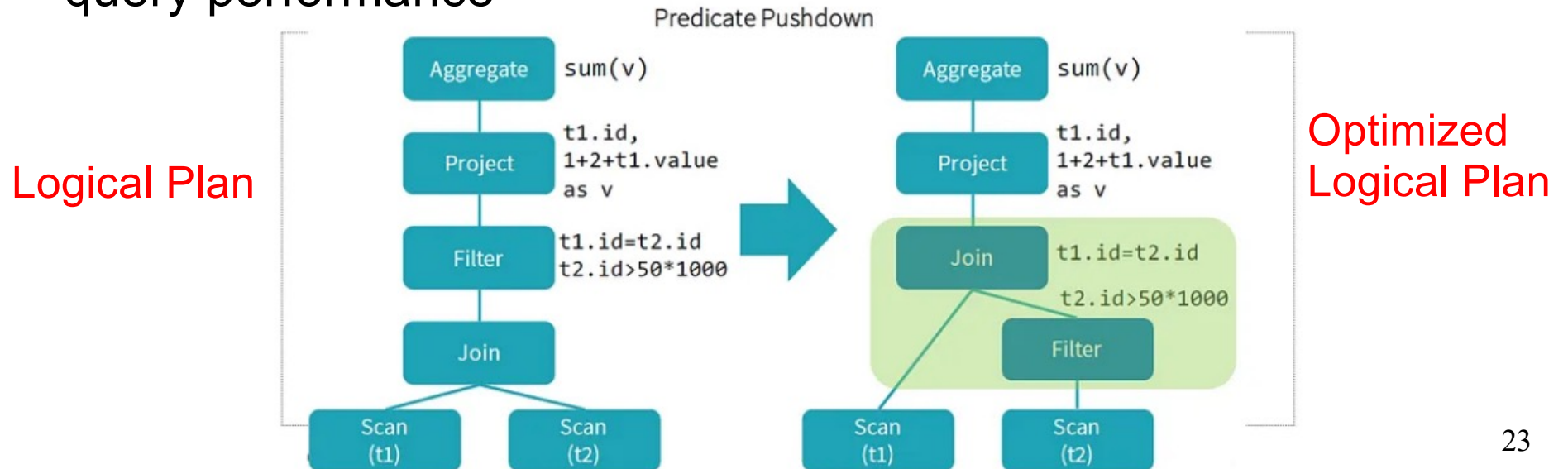
Reduces entries retrieved from database and improves
query performance

By default, Spark will automatically push down
valid WHERE clauses to database

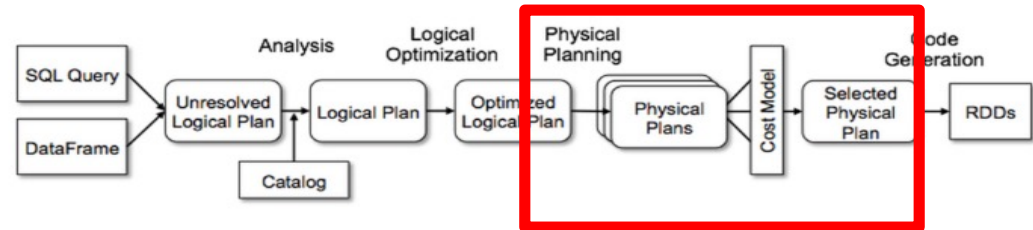
Predicate Pushdown example

```
SELECT sum(v)
FROM (
  SELECT
    t1.id,
    1 + 2 + t1.value AS v
  FROM t1 JOIN t2
  WHERE
    t1.id = t2.id AND
    t2.id > 50 * 1000) tmp
```

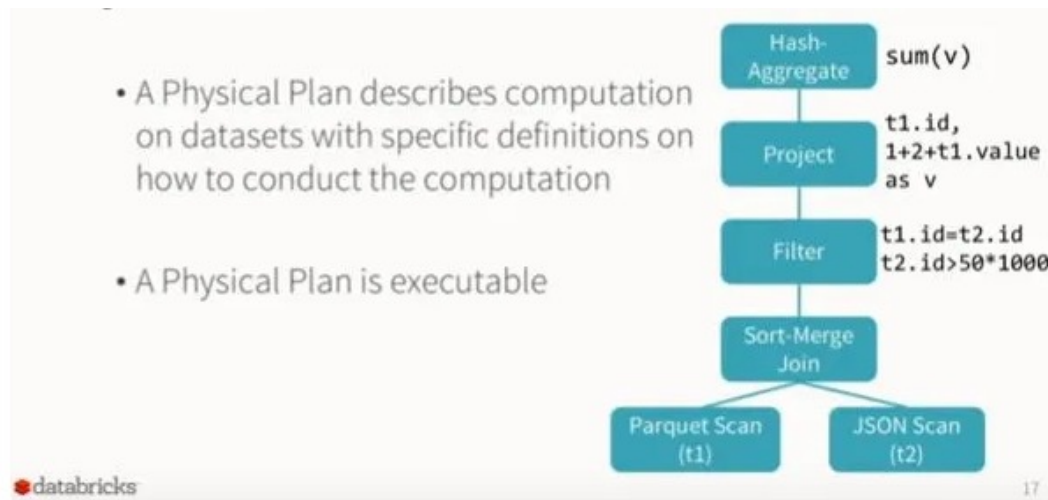
- Notice two conditions in WHERE
- Can apply filter before doing the JOIN
- Reduces entries retrieved from database and improves query performance



Physical Plan

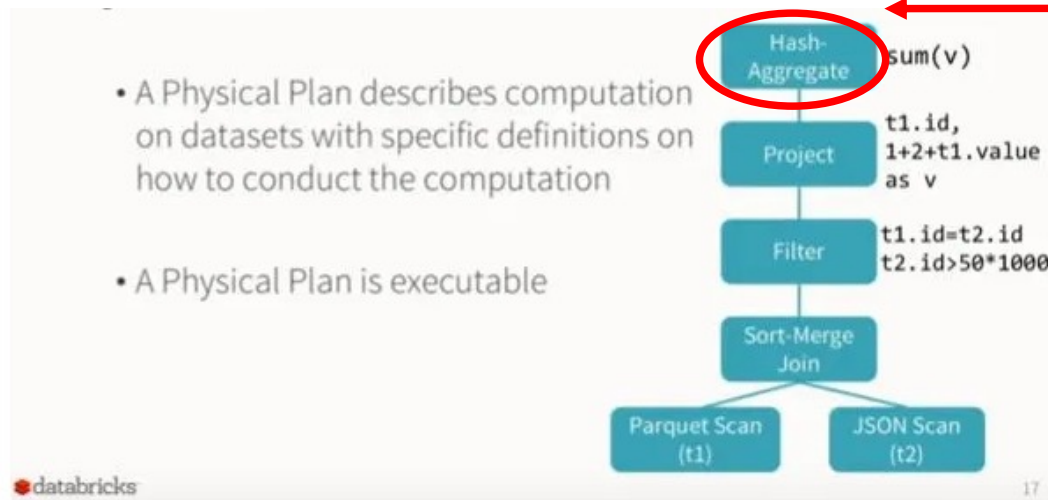
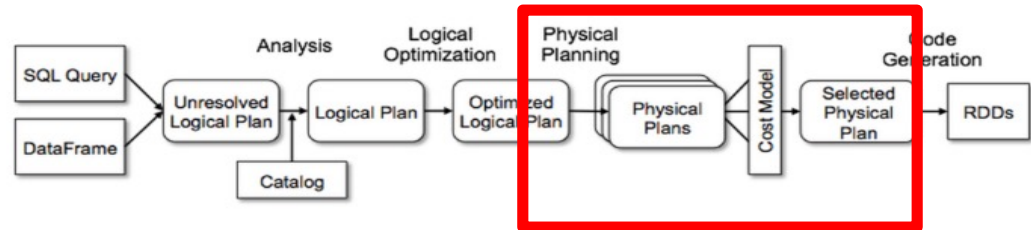


Spark takes logical plan and generates physical operators matching execution engine



Optimized Logical Plan to Physical Plan

Physical Plan



For aggregation, Spark decides between **hash aggregation** and **sort-based aggregation**

Creating hash map in memory works better for small number of groups

As rows are processed, Spark uses hash map to accumulate aggregate values for each group

Optimized Logical Plan to Physical Plan

Physical Planning and Cost Model

What kinds of decisions can be made?

type of join, sequencing of filters, for example

Selects best plan based on *cost model*

Physical Planning and Cost Model

What kinds of decisions can be made?

type of join, sequencing of filters, for example

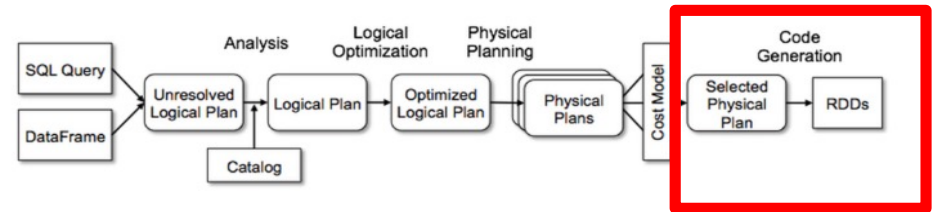
Selects best plan based on *cost model*

Paper: A Cost Model for SPARK SQL (Golfarelli & Baldacci)

- Takes into account network, CPU, and IO costs
- Set of Spark operations are analytically modeled based on the cluster and application parameters, database statistics
- Experimental results show that execution time estimates have average error of 14-20%; good enough to be useful

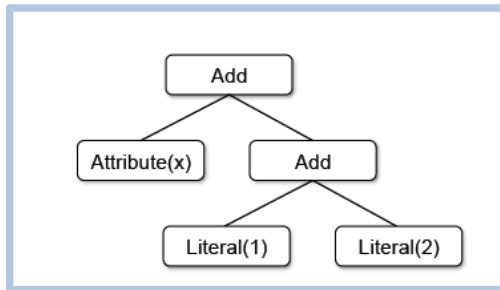
Code Generation

Code Generation



Consider this earlier calculation:

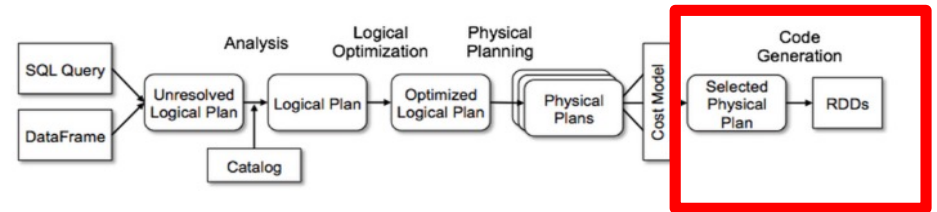
$x + (1 + 2)$



Without code generation, this would need to be interpreted for each row of data

With code generation, can translate to something that can be compiled

Code Generation



Strategy:

1) Convert SQL → Abstract Syntax Tree (AST)

Abstract because superficial things like spaces don't matter

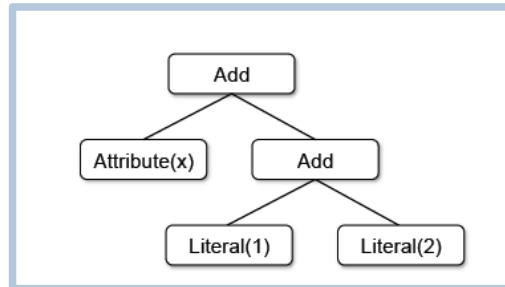
An AST is a tree representation of code. It is fundamental to compilers.

2) Have Scala evaluate the AST

3) Compile and run the generated code

Code Generation Example

$x + (1 + 2)$



Function to translate tree to Scala AST

```
def compile(node: Node): AST = node match {  
  case Literal(value) => q"$value"  
  case Attribute(name) => q"row.get($name)"  
  case Add(left, right) => q"${compile(left)} + ${compile(right)}"  
}
```

Strings with **q** are called *quasiquotes*

Quasiquotes allow programmatic construction of ASTs in Scala
which can be fed to Scala compiler

Evaluation Setup

Cluster of six EC2 i2.xlarge machines (one master, five workers)

Each with 4 cores, 30 GB memory, 800 GB SSD

running HDFS 2.4, Spark 1.3 (note this is a very early version)

Dataset was 110 GB of data after compression using columnar Parquet format

Evaluation

Dataset consists of 1 billion integer pairs (a, b) with 100,000 distinct values of a

Task: measure time taken to compute the average of b for each value of a .

Evaluation, contd.

Approach 1: RDDs

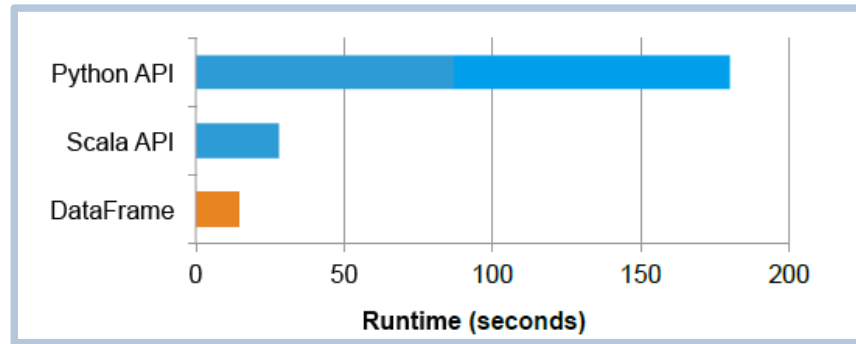
```
sum_and_count = \  
    data.map(lambda x: (x.a, (x.b, 1))) \  
        .reduceByKey(lambda x, y: (x[0]+y[0], x[1]+y[1])) \  
        .collect()  
[(x[0], x[1][0] / x[1][1]) for x in sum_and_count]
```

Approach 2: DataFrame API (from Spark SQL)

```
df.groupBy("a").avg("b")
```

- > DataFrame API code is more concise
- > Also much faster as physical execution is compiled as JVM bytecode

Evaluation, contd.



- > **Python API:** means RDD version, not optimized by Catalyst.
- > **Scala API:** Scala runs in the JVM. Programmer supplies arbitrary Scala functions. Spark has limited ability to understand and optimize what those functions are doing.
- > **DataFrame:** Tells Spark what you want, not how to do it. Catalyst fully optimizes.

Sources

Spark SQL: Relational Data Processing in Spark. Armbrust et. al.

<https://www.databricks.com/blog/2015/04/13/deep-dive-into-spark-sqls-catalyst-optimizer.html>

https://medium.com/@Shkha_24/catalyst-optimizer-the-Power-of-spark-sql-cad8af46097f

https://docs.datastax.com/en/dse/6.8/dse-dev/datastax_enterprise/spark/sparkPredicatePushdown.html

<https://www.twilio.com/blog/abstract-syntax-trees#:~:text=In%20general%20an%20AST%20is,that%20represents%20a%20function%20call.>

<https://www.techtarget.com/whatis/definition/bytecode#:~:text=An%20example%20of%20bytecode&text=When%20an%20application%20is%20written,running%20on%20a%20target%20system.>